

Proyecto 7: Terminal Web Retro

Curso RTOS Raiz

Integrantes:

Juan Diego Restrepo Hernández

Santiago Galeano Castaño

10 de diciembre de 2025

1. Resumen

Este proyecto implementa un servidor web retro estilo terminal en un **ESP32** que permite el control de hardware mediante una interfaz web moderna con estética estilo Matrix. El sistema opera como un **Access Point (SoftAP)** independiente. Proporciona una **Autenticación Segura** con sesiones basadas en IP y timeout automático, permitiendo el **Control de Hardware** (LEDs amarillo y azul mediante GPIO) y la lectura de un **Sensor de Temperatura NTC**. El frontend web está almacenado en una partición **SPIFFS**. La arquitectura subyacente utiliza el kernel **FreeRTOS** con tareas concurrentes y **Colas de Mensajes** para la sincronización y el procesamiento asíncrono de comandos.

Palabras Clave: ESP32, FreeRTOS, RTOS, SoftAP, Servidor Web Embebido, Terminal Retro, SPIFFS, GPIO, Sensor NTC, Autenticación.

2. Introducción

El objetivo principal de este proyecto es la implementación de un servidor web con una interfaz de terminal de estilo retro, inspirado en Matrix, sobre la plataforma de un microcontrolador **ESP32**. Esta solución está diseñada para permitir el control de periféricos de hardware de bajo nivel mediante una interfaz de usuario moderna y estilizada.

El sistema se distingue por sus características de autonomía y seguridad: opera como un

Access Point (SoftAP), lo que le permite funcionar en entornos sin infraestructura de red preexistente. Además, implementa un robusto sistema de **Autenticación Segura** basado en la dirección IP del cliente y un mecanismo de gestión de sesiones con timeout por inactividad.

La funcionalidad principal radica en el **Control de Hardware**, donde el usuario puede interactuar con LEDs a través de pines GPIO y obtener lecturas en tiempo real de un sensor de temperatura NTC. Todo el frontend web (HTML/CSS/JS) está convenientemente almacenado en la partición **SPIFFS** del flash del ESP32.

Finalmente, el proyecto sirve como un caso de estudio de la arquitectura **FreeRTOS**, aprovechando sus tareas concurrentes y colas de mensajes para el manejo eficiente y asíncrono de las peticiones HTTP y las operaciones de control de GPIO.

3. Planteamiento del Problema

La necesidad primordial es desarrollar un ****sistema autónomo y robusto**** para el control remoto de hardware aprovechando las capacidades de un Sistema Operativo en Tiempo Real (**RTOS**). El desafío central se enfoca en gestionar de manera eficiente las ****operaciones concurrentes****, tales como el manejo de múltiples peticiones HTTP, la lectura periódica de datos de sensores y la ejecución de comandos GPIO, sin que estas acciones bloqueen el fun-

cionamiento del sistema en red.

Para lograr un sistema útil y seguro, se identifican los siguientes problemas específicos:

1. **Conectividad Autónoma:** ¿Cómo se puede garantizar el acceso al sistema en entornos sin una red WiFi externa, y cómo se gestionan múltiples clientes de manera simultánea?
2. **Seguridad y Exclusividad:** ¿Cómo implementar un mecanismo de **autenticación** seguro (basado en sesiones y timeouts) para evitar el control no autorizado del hardware?
3. **Interacción Asíncrona:** ¿Cómo desacoplar la lógica de red (servidor HTTP) de la lógica de control de hardware (GPIO), para asegurar que las operaciones de bajo nivel no retrasen las respuestas del servidor y viceversa?
4. **Persistencia de Interfaz:** ¿Cómo se puede almacenar la interfaz de usuario web de manera eficiente en el dispositivo embebido (**ESP32**) para que el sistema sea completamente autocontenido?

4. Objetivos

4.1. Objetivo General

Implementar un sistema de control de hardware embebido, autónomo y seguro, basado en la plataforma **ESP32** y el sistema operativo **FreeRTOS**, con una interfaz de usuario web de estilo retro.

4.2. Objetivos Específicos

1. Desarrollar y configurar el **ESP32** como un punto de acceso (**SoftAP**) independiente, garantizando la conectividad directa sin necesidad de una red externa.
2. Diseñar e implementar un **Servidor HTTP** embebido que sirva la interfaz de usuario web alojada en la partición **SPIFFS**.

3. Establecer un mecanismo de **Autenticación** y gestión de sesiones robusto que incluya verificación de credenciales, sesiones basadas en IP y un mecanismo de **timeout** por inactividad.
4. Utilizar las primitivas de **FreeRTOS** (tareas concurrentes, colas de mensajes y mutexes) para desacoplar el procesamiento de comandos de hardware de la lógica de red.
5. Implementar la funcionalidad de control de hardware mediante comandos, incluyendo la activación/desactivación de **LEDs** por GPIO y la lectura continua de un **Sensor de Temperatura NTC**.

5. Alcance

El alcance del proyecto está delimitado a la implementación y validación de las siguientes funcionalidades clave, centradas en la plataforma **ESP32** con **FreeRTOS**:

5.1. Funcionalidades Incluidas

1. **Autonomía de Red:** El sistema operará exclusivamente como un **Access Point (SoftAP)**, proveyendo su propia red WiFi para la conexión de clientes.
2. **Persistencia de Interfaz:** Se cargará y servirá toda la interfaz web (HTML, CSS, JavaScript) desde la partición **SPIFFS** del ESP32, asegurando que el dispositivo sea completamente autocontenido.
3. **Control de Hardware:** El sistema manejará las operaciones de encendido y apagado de los LEDs Amarillo y Azul a través de comandos específicos de terminal web.
4. **Monitoreo:** Se implementará la lectura de la temperatura ambiente utilizando un **Sensor NTC**, cuyos datos serán accesibles a través de un ***endpoint*** HTTP.
5. **Gestión de Sesiones:** La seguridad se limitará a la autenticación mediante usuario/contraseña y la gestión de sesiones basada en la IP del cliente con un mecanismo de ***timeout*** por inactividad.

5.2. Funcionalidades Excluidas

El proyecto no contempla las siguientes funcionalidades, que podrían ser consideradas en futuras iteraciones:

- **Conexión a Red Externa:** El sistema no funcionará como cliente (**STA**) para conectarse a una red WiFi existente.
- **Cifrado de Comunicaciones:** No se implementarán protocolos de seguridad a nivel de transporte como **HTTPS/SSL/TLS**.
- **Múltiples Usuarios:** El sistema está diseñado para un único par de credenciales (**root/matrix123**) y no incluye gestión de roles de usuario.
- **Persistencia de Estado:** Los estados de los LEDs y las sesiones no se guardan en memoria no volátil, y se restablecen tras un reinicio del sistema.

6. Arquitectura del Sistema

El diseño del sistema está centrado en la utilización eficiente de los recursos del **ESP32** mediante el kernel **FreeRTOS**. Esto permite la ejecución de múltiples procesos de forma concurrente y la comunicación segura entre ellos.

6.1. Componentes Principales

La arquitectura se divide en tres capas fundamentales:

- **Capa de Red (SoftAP y HTTP Server):** Gestiona la conectividad autónoma del dispositivo y el manejo de peticiones HTTP, incluyendo la autenticación y la gestión de sesiones.
- **Capa de Aplicación (FreeRTOS Tasks):** Compuesta por tareas dedicadas (p. ej., control GPIO, gestión de sesiones, lectura de sensores) que operan de forma asíncrona.

- **Capa de Persistencia (SPIFFS):** Almacena la interfaz de usuario web (*frontend* estilo terminal retro), permitiendo que el sistema sea *standalone*.

6.2. Flujo de Control Asíncrono

La comunicación entre la capa de red y la capa de aplicación se logra mediante **Colas de Mensajes (Queues)** de FreeRTOS. Cuando un usuario envía un comando (**led y on**), el servidor HTTP no ejecuta la acción directamente. En su lugar, envía el comando a una cola (**gpio_command_queue**). Una tarea RTOS dedicada consume esta cola, ejecuta la acción de hardware y envía una respuesta de vuelta a través de otra cola (**gpio_response_queue**), logrando un alto grado de desacoplamiento y evitando el bloqueo del servidor.

6.3. Diagrama de Arquitectura

A continuación, se presenta un diagrama simplificado que ilustra la orquestación de las tareas de FreeRTOS y los componentes del sistema.

6.4. Diagrama de Arquitectura

A continuación, se presenta un diagrama que ilustra la orquestación de las tareas de FreeRTOS y el flujo de comunicación asíncrona mediante colas.

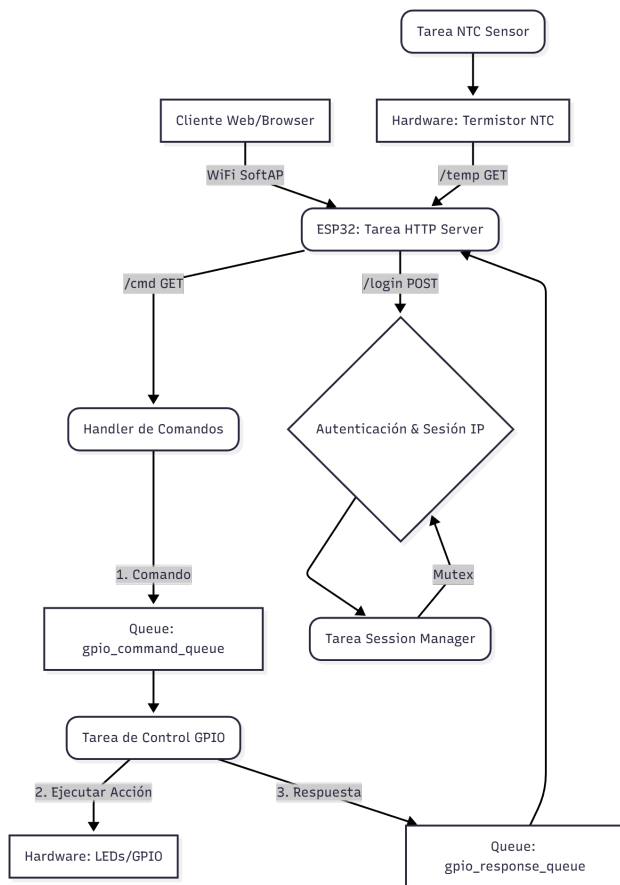


Figura 1: Diagrama de la arquitectura concurrente basada en FreeRTOS y colas de mensajes.

7. Descripción del Hardware

El proyecto se basa en los siguientes componentes de hardware interconectados. El **ESP32** es el núcleo del sistema, responsable de ejecutar FreeRTOS y el servidor web. Provee la capacidad de red **WiFi** para el modo SoftAP.

7.1. Componentes

- **Microcontrolador: ESP32** (Módulo WROOM o similar).
- **Actuadores: Dos LEDs** (Amarillo y Azul) conectados a pines **GPIO** configurados como salidas (GPIO2 y GPIO4).
- **Sensor: Termistor NTC 10k.** Utilizado para la lectura de temperatura ambiente.

- **Conversor Analógico-Digital (ADC):** El sensor NTC se conecta al **ADC1, Canal 4 (GPIO32)** del ESP32, mediante un divisor de tensión (con una resistencia fija de pull-up) para el cálculo de la temperatura.

7.2. Diagrama de Conexión

El siguiente diagrama muestra la disposición física de los componentes y su conexión a los puertos GPIO específicos del microcontrolador.

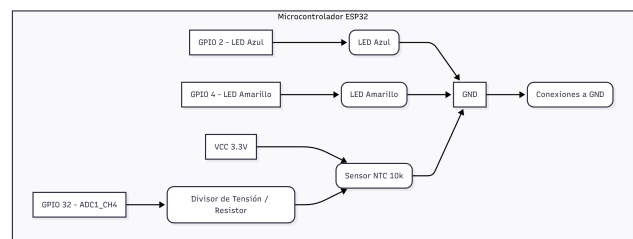


Figura 2: Esquema de conexiones del hardware. El NTC requiere un divisor de tensión para su lectura por ADC.

7.3. FreeRTOS: Tareas y Sincronización

El sistema opera con múltiples tareas concurrentes, cada una dedicada a una función específica. La comunicación entre el servidor HTTP y la lógica de hardware se realiza de forma asíncrona mediante **Colas de Mensajes** (*Queues*), y la protección de recursos se garantiza con **Mutexes**.

Las tareas principales son:

- **Tarea HTTP Server:** Maneja todas las peticiones de red y la autenticación.
- **Tarea GPIO Command:** Ejecuta acciones de bajo nivel (LEDs) al recibir mensajes de la cola de comandos.
- **Tarea Session Manager:** Monitorea los timeouts por inactividad.
- **Tarea Sensor NTC:** Lee periódicamente la temperatura y prepara los datos para el servidor.

7.4. Diagrama de Flujo de Software

El siguiente diagrama ilustra cómo las tareas se comunican y cómo se procesa un comando desde el cliente hasta el hardware y de vuelta.

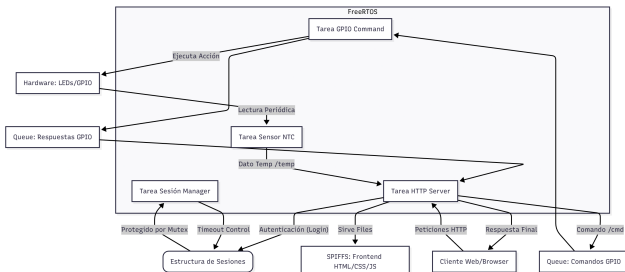


Figura 3: Diagrama de flujo de datos y control basado en FreeRTOS y sus primitivas de sincronización.

8. Conclusiones

El proyecto "Terminal Web Retro" ha demostrado ser una implementación exitosa y robusta de un servidor web embebido y autónomo sobre la plataforma **ESP32** con el sistema operativo **FreeRTOS**.

- **Validación de Arquitectura RTOS:** Se validó la eficacia de la arquitectura basada en **FreeRTOS**. El uso de tareas concurrentes y **Colas de Mensajes** permitió un desacoplamiento efectivo entre la capa de red (Servidor HTTP) y la capa de control de hardware (GPIO), garantizando que las operaciones de bajo nivel se ejecuten de forma asíncrona sin bloquear la respuesta del servidor o la conectividad WiFi.
- **Autonomía y Portabilidad:** La configuración como **SoftAP** y el almacenamiento del *frontend* web completo en la partición **SPIFFS** lograron el objetivo de crear un sistema completamente autónomo e independiente de una infraestructura de red externa, validando su portabilidad.
- **Seguridad Funcional:** La implementación del sistema de **Autenticación** con

sesiones basadas en IP y **timeout** de inactividad proporciona una capa de seguridad esencial, limitando el acceso al control de hardware solo a usuarios previamente autenticados dentro del tiempo de la sesión.

- **Funcionalidad de Control:** Los comandos de control de **LEDs** y la lectura del **Sensor NTC** se integraron exitosamente, demostrando la capacidad del sistema para interactuar de manera confiable con periféricos de entrada/salida y lectura analógica en tiempo real.

En resumen, el proyecto cumple con todos los objetivos planteados, sirviendo como una prueba de concepto sólida para el desarrollo de dispositivos IoT seguros y basados en RTOS.